STUDY NOTES

User Stories Applied

A chapter-by-chapter summary of Mike Cohn's book on agile requirements

Book: *User Stories Applied: For Agile Software Development*, Mike Cohn, Addison-Wesley, 2004

These notes: an original summary written for study. The ideas are the author's; the wording is not. Short quotations are marked and attributed.

About these notes

These are study notes on *User Stories Applied: For Agile Software Development* by Mike Cohn (Addison-Wesley, 2004). They follow the book's own order: five parts, twenty-one chapters and an appendix on Extreme Programming.

Everything here is a summary in my own words, written to help a reader recall and apply the ideas. It is not a substitute for the book, which contains the full arguments, the worked examples, the figures, and end-of-chapter questions with answers. A handful of short quotations are marked and given a page number from the printed edition. Where an example is not the book's own, it is labelled as mine.

Each chapter section has the same shape: a one-line summary, the main ideas, and a takeaway. A glossary and two quick-reference tables are at the end.

The book in one page

If you only remember ten things from the book, these are the ones to keep.

- **Requirements are a communication problem.** The people who want software and the people who build it see the world differently. A method works when neither side dominates and both share the trade-off between scope and time.
- **A story has three parts:** a short written reminder (the card), the discussion that fills in detail (the conversation), and the tests that record what was agreed (the confirmation). The card is the least important of the three.
- **Good stories pass the INVEST check:** independent, negotiable, valuable to users or purchasers, estimatable, small, testable.
- **Write stories for specific kinds of user.** Model the user roles first; it stops the team from designing for one imaginary average person.
- **Real users beat stand-ins.** When you must use a proxy (a manager, a salesperson, an analyst), know that person's bias and compensate for it.
- **The customer owns priority; the developers own estimates.** Neither may do the other's job.
- **Estimate in relative units (story points) as a team,** then let measured velocity turn points into calendar time.
- **Plan at two levels.** A release plan is a rough allocation of stories to iterations. An iteration plan breaks the next few stories into tasks that individuals sign up for.

- **Count only finished work.** Velocity is the points of stories that pass their acceptance tests. Half-done stories count as zero.
- **Defer detail until it is needed.** Distant work stays as large placeholder stories; near work is split small. Expect the plan to change and re-plan every iteration.

Part I: Getting Started

The first seven chapters cover what a story is, how to write one, who it is for, how to find stories, who can speak for users, how stories are tested, and a set of writing guidelines.

Chapter 1. An Overview

In brief: the whole method in miniature: what a story is, where the detail lives, and how stories drive planning and testing.

The problem stories solve

A software project depends on two groups understanding each other: those who want the product (customers, users, analysts, domain experts) and the technical team. When the business side controls the exchange, scope and dates get dictated with little regard for whether both can be met. When the technical side controls it, jargon takes over and developers stop learning what is needed. Since no project can be predicted in full at the start, the sensible response is to make many small decisions throughout, using a process that surfaces information early and often. Stories are the tool for that.

Card, conversation, confirmation

A user story describes a piece of functionality that matters to someone who uses or buys the system. It has three aspects, named by Ron Jeffries:

- **Card:** one or two sentences, traditionally handwritten on an index card, used for planning and as a reminder.
- **Conversation:** the discussions between customer and developers where the details get worked out.
- **Confirmation:** the tests that capture those details and show when the story is done.

The book quotes Rachel Davies on the role of the card:

[Cards] "represent customer requirements rather than document them."

— Rachel Davies, quoted on p. 4

What makes something a story

- It must be valued by a user or purchaser. A statement about which programming language or database technique to use is not a story for an ordinary business application, because its users do not care.
- Very large stories are called **epics**. They are fine as placeholders but must be split before they are built.
- As a starting guide, a story should take one programmer or one pair somewhere between half a day and about two weeks.

- Do not keep splitting just to record every detail. Details belong in the conversation, and the ones that are agreed get noted as short test reminders on the back of the card.
- Stories are not contracts. Nobody should be able to point at a card months later as proof of what was promised; the tests are the record of agreement.

How a story-driven project runs

- A **customer team** (product manager, real users, testers, interaction designers) writes the stories, because stories must be in business language and the customer has to prioritise them.
- An initial batch usually comes out of a story-writing workshop.
- Developers estimate each story.
- Customer and developers pick an iteration length, typically one to four weeks, and keep it fixed.
- Developers guess their **velocity**: how much estimated work fits in one iteration. The first guess will be wrong and gets replaced by measurement.
- A **release plan** is made by filling iterations in priority order without exceeding velocity.
- Each iteration delivers working code for its stories; the customer specifies tests during the iteration and adjusts the plan before the next one.

If the next story in priority order does not fit in what remains of an iteration, there are two options: pull forward a smaller, lower-priority story, or split the large story and take its most important part.

Why change to stories

The chapter previews the case made in Chapter 13: stories favour talking over writing, both sides can read them, they are a convenient unit for planning, they suit iterative work, and they let a team postpone detail until it knows the feature is wanted.

Takeaway A story is a promise to have a conversation. The card reminds you to talk; the tests record what you decided.

Chapter 2. Writing Stories

In brief: six attributes of a good story, remembered by Bill Wake's acronym INVEST, plus how to split and combine stories.

Attribute	What it means	What to do when it fails
Independent	Stories can be built in any order, so they can be prioritised and estimated separately.	Merge the dependent stories into one, or split along a different line. As a last resort write two estimates on the card (done first / done later).
Negotiable	The card is a reminder, not a specification. Detail is agreed in conversation.	Strip excess notes from the front of the card. Keep only the story and open questions; move agreed details to tests on the back.
Valuable	Someone who uses or pays for the software can see why they want it.	Rewrite technology-centred stories in terms of the benefit. Best fix: let the customer write the stories.
Estimatable	Developers can make a reasonable guess at size.	Missing domain knowledge: ask the customer. Missing technical knowledge: run a time-boxed experiment (a spike). Too big: split it.

Small	Fits comfortably in an iteration, yet is not so tiny that writing it down costs more than doing it.	Split epics; staple tiny items (bug fixes, small UI tweaks) together into one story.
Testable	There is a way to show it is finished.	Replace vague words with a measurable target, e.g. a response time met in a stated percentage of cases.

Points worth remembering

- **Too much detail is harmful, not just wasteful.** A card crammed with notes looks precise, so people assume no conversation is needed and treat rough estimates as exact.
- **Users and purchasers are different people.** A buyer may value things no user notices, such as central configuration or compliance with an audit standard. Those are legitimate stories. Stories valued only by developers are not.
- **Two kinds of epic.** A *compound* story hides several smaller stories and can be split, commonly by operation (create, edit, delete) or by the data involved. A *complex* story is inherently large or uncertain; split it into an investigation with a fixed time limit and a second story for the real work.
- **Put the spike in an earlier iteration than the work it informs** where you can, so that the customer can prioritise research separately from the feature and the iteration is not filled with work nobody can size.
- **Aim to automate nearly all tests.** A few (for example, whether novices can use a feature untrained) need human observation, but they are the exception.
- **"Never" and "always" are untestable.** State how often and how fast instead.

Takeaway Run each story through INVEST. When one letter fails, the fix is almost always to split, combine, or reword in terms of user value.

Chapter 3. User Role Modeling

In brief: identify the distinct kinds of user before writing stories, so the product is not built for a single imagined user.

Teams often write every story from one viewpoint and so miss the needs of anyone who does not fit it. Borrowing from usage-centred design (Constantine and Lockwood) and interaction design (Cooper), the chapter recommends a short role-modeling session. A **user role** is a set of characteristics shared by a group of users and the way they will interact with the system.

The four steps

- **Brainstorm.** Customer and developers write role names on cards and place them on a table or wall, saying only the name. No discussion or judging. About fifteen minutes is usually enough.
- **Organise.** Move the cards so that overlapping roles physically overlap, by as much as the roles do.
- **Consolidate.** Discuss overlapping cards; merge or tear up duplicates. Also discard roles that will not affect the product's success.
- **Refine.** Add distinguishing attributes to each remaining card.

Useful attributes to note for each role: how often they will use the software, how well they know the domain, their general comfort with computers, their skill with this particular product, and what they mainly want from it (speed, convenience, a rich experience, and so on).

Guidance

- A role should stand for one person, not an organisation. A company cannot use software; a recruiter at that company can.
- Keep roles human where possible. Add another system as a role only if it helps the team think.
- The whole exercise should take about an hour, and most teams should stop there.
- Do it even when real users sit nearby. Available users are not necessarily the right mix of users.

Two optional extras

- **Personas.** For one or two roles that the product must satisfy, invent a specific person with a name, a photo and enough background to feel real, and write stories about that person. Make sure the persona reflects actual market research.
- **Extreme characters.** Imagine how wildly unusual people would use the product. It may reveal stories you would otherwise miss, though there is no easy way to tell whether such stories belong in the product, so spend little time on it.

Takeaway Replace “the user” in your stories with a named role. It costs an hour and changes which stories get written.

Chapter 4. Gathering Stories

In brief: requirements are not lying around waiting to be collected; you go looking for them with several techniques, repeatedly.

Trawling, not capturing

Words such as “elicit” and “capture” suggest that users already hold every requirement and that, once written down, requirements stay put. The chapter prefers the Robertsons' image of **trawling** with a net. It fits in four ways: nets of different mesh catch requirements of different sizes, so you can make a coarse first pass and finer ones later; requirements grow in importance or die as the product evolves; you never catch everything, and you catch some rubbish; and skill affects the catch.

How much to gather up front

Agile teams accept that one pass cannot find everything. Still, make an early attempt: look ahead roughly one release (three to six months) and write what you can see, with less detail the further away the work is. A one-line placeholder is enough for something the customer “probably” wants later. This early pass also gives the rough size needed for budgeting and approval.

Four techniques

Technique	Good for	Watch out for
User interviews	The default. Talk to real users across different roles.	Users are poor at stating what they need and are not automatically the best people to design the solution.

Questionnaires	Getting answers from many users to specific questions; helping to prioritise stories you already have.	One-way and slow, with no follow-up questions. Not suitable as the main way to find new stories.
Observation	Seeing what people really do, which often differs from what they say they need.	Opportunities are rare outside in-house projects. Releasing early creates them.
Story-writing workshops	The fastest way to produce many stories. Hold one before each release.	Keep it high level: no screen design, no debating individual stories.

One user's remark, quoted in the chapter, sums up why asking "what do you need?" is not enough:

"You built exactly what I asked for but it's not what I want."

— a user, quoted on p. 46

Asking good questions

- **Open-ended:** invites more than yes or no. "What would you give up to get X?" reveals trade-offs that "Would you like X?" hides.
- **Context-free:** does not suggest its own answer or presume a requirement exists. Asking how fast a feature must be implies that there is a speed requirement; asking what performance matters, and where, does not.
- Start broad and move to specific questions later.

Running a workshop

- Sketch a rough, low-fidelity map of the product on paper or a whiteboard: boxes for areas of the product, arrows for how a user moves between them. It shows workflow, not screens.
- Take one role or persona at a time. Starting from an empty box for the main screen, ask what that user can do from there; each answer becomes a new box and a story.
- Go depth-first: follow one path to its end before returning to explore siblings.
- Prompt for gaps: what would the user do next, what mistakes could they make, what might confuse them, what else would they need to know?
- Favour quantity over quality. Weak stories can be torn up or given low priority later.
- Keep a list of issues to return to, notice who is silent, and throw the sketch away within a few days.

Takeaway Use several techniques, not one. Ask open, context-free questions, and write stories at a level of detail that matches how soon they will be built.

Chapter 5. Working with User Proxies

In brief: when real users are unavailable, someone stands in for them. Each kind of stand-in distorts the picture in a predictable way.

Proxy	Typical bias or limitation
Users' manager	Uses the product differently from front-line staff, if at all, and may exaggerate problems as they are passed up the chain. Check their claims with real users.

Development manager	Among the worst choices: likely to favour interesting technology or a bonus-linked date, and rarely has hands-on experience as a user.
Salesperson	Prioritises whatever feature lost the most recent sale. Valuable as a route to meeting customers.
Domain expert	Excellent for business rules and the domain model, but may steer the product toward experts like themselves and away from ordinary users.
Marketing group	Understands markets more than users; tends to value the number of features over their quality. Useful for relative priorities.
Former user	Good if the experience is recent. Check that their goals still match those of current users.
Customer (the buyer)	Decides the purchase but may never use the product. Their needs, such as security, are real but differ from users' needs.
Trainers and support staff	Will push for a product that is easy to train or easy to support, which is not the same as what users want.
Business or systems analyst	Often a good proxy, with a foot in both worlds, provided they talk to users instead of reasoning alone, and do not over-invest in up-front analysis.

Making the best of a proxy

- **Users exist but access is blocked:** ask to set up a *user task force* of real users as a sounding board. The proxy keeps the final say on direction; day-to-day detail increasingly comes from the task force.
- **No users are reachable:** use more than one proxy, of different kinds, so the product is not shaped by one person's needs.
- **Study competing products:** reviews and user forums show which features matter and which are too complicated.
- **Release early,** even as a beta, to find where the proxy's view and real users' views differ and to open a line to those users.
- **Do not appoint yourself.** Developers are usually worse stand-ins than any of the proxies listed.

Building the customer team

- Add real users, ideally one for each important type of user.
- Name one person as project champion who coordinates the team and makes sure it gives a single, consistent message.
- Identify what the project's success depends on, and add proxies whose knowledge covers those factors.

"While there may not be one customer, there must be one customer voice."

— p. 64

Takeaway A real user is always better than a proxy. If you must use proxies, pick several kinds so their biases offset one another.

Chapter 6. Acceptance Testing User Stories

In brief: acceptance tests hold the detail that a story card leaves out, and they define when a story is finished.

What the tests are for

- They record details that came out of customer and developer conversations, in place of long lists of formal requirement statements.
- They expose assumptions the customer holds but has not said aloud.
- They give a clear finishing line, which stops the team from doing too little or too much.

How testing works with stories

- **Two stages.** First jot short test reminders on the back of the card whenever one occurs to someone. Later, turn them into full tests.
- **Before coding.** Tests written ahead of programming tell the programmer about cases they might otherwise forget. They are added during conversations, in a focused session at the start of each iteration, and whenever new cases come to light.
- **The customer specifies them,** with help from testers and programmers, because the software exists to realise the customer's vision. Developers may add tests of their own.
- **Testing is part of development,** not a phase after it. A tester who learns the system only from programmers ends up checking that the code does what the programmer said, not what users want.
- **Know when to stop.** Write tests while they add clarity. The customer need not cover every low-level case; programmers' unit tests should do that.

Prompts that help the customer find tests: is there anything further the programmers should be told, which of my assumptions about the implementation are unstated, in what circumstances would the behaviour differ, and what could fail?

Automation

The customer should run the acceptance tests at least at the end of every iteration, including those from earlier iterations, since new work can break old work. That grows expensive, so automate where possible. The book recommends Ward Cunningham's Framework for Integrated Test (FIT) and FitNesse, which builds on it: the customer writes inputs and expected results as rows in a table, a programmer writes the small amount of glue code, and results show as green or red cells.

Other kinds of testing

Story tests are mostly functional. Depending on the product, also plan for user-interface testing, usability testing, performance testing and stress testing. The aim on an agile project is to find and remove defects using judgement and experience, without chasing complete coverage and without blame when something slips through.

Takeaway Tests are how stories carry detail. Write them early, have the customer own them, and automate them so they can run every iteration.

Chapter 7. Guidelines for Good Stories

In brief: thirteen practical habits for writing better stories.

- **Start from goals.** For each role, list what that user is trying to achieve with the product; those goals are high-level stories that lead to others.
- **Slice the cake.** When splitting, make each piece run through every layer of the system so it delivers something usable. Avoid splits along technical lines such as "the form" and "the database write", which are useless on their own.
- **Write closed stories.** Each should end with the user having achieved something definite. "Manage" is a warning word because managing never finishes.
- **Use constraint cards.** Rules the system must obey (capacity, platforms, uptime) are written on a card marked as a constraint. They are not scheduled like stories, but should be backed by an automated test where possible.
- **Size to the horizon.** Small, specific stories for the next few iterations; broad ones for later.
- **Keep the user interface out for as long as you can.** Stating a solution inside a requirement closes off design options. UI detail creeps in naturally once the product exists.
- **Not everything is a story.** Use another format (a UI guideline document, an interface specification) where it fits better.
- **Name the role.** One widely used template, from the XP team at Connextra, has three slots: the role, what they want, and the business reason.
- **Write for one user.** Singular phrasing exposes ambiguities, such as whether a person may delete only their own records.
- **Use the active voice.**
- **The customer writes.** Developers may help or suggest, but responsibility stays with the customer, who must understand each story to prioritise it.
- **Do not number the cards.** Numbers add overhead and make discussion abstract. Give a card a short title if you need a handle.
- **Remember the purpose.** A card is a reminder to talk. Add only enough to pick the conversation up again.

My example of slicing (not from the book): for a library app, "A member can reserve a book" is too big. A poor split is "build the reservation screen" and "store reservations". A better split is "A member can reserve a book that is on the shelf" and "A member can join a waiting list for a book that is out on loan". Each works end to end.

Takeaway Split so that every piece delivers value on its own, name the role, and keep the solution out of the requirement.

Part II: Estimating and Planning

Sooner or later someone asks when the work will be finished. These four chapters explain how to size stories, build a release plan, plan a single iteration, and track progress.

Chapter 8. Estimating User Stories

In brief: estimate stories in relative units called story points, as a team, and let velocity convert points into time.

What a good estimating approach needs

It should let you revise an estimate when you learn more, work for both epics and small stories, be quick, tell you something useful about progress and remaining work, tolerate imprecision, and support release planning.

Story points

- Each team defines its own unit. It may be an ideal day, an ideal week, or a measure of complexity.
- Cohn's preference is the **ideal day**: a day of work with no meetings, email or interruptions. It is easier to think in than elapsed time and easier to convert to a schedule later than a purely abstract unit.
- An estimate should cover everything needed to finish the story, including testing and time spent with the customer.

Estimating as a team

Story estimates belong to the whole team, because nobody yet knows who will do the work and because group estimates are more reliable. The method is an iterative one derived from Wideband Delphi:

- The customer reads out a story and answers the developers' questions.
- Each developer privately writes an estimate on a card.
- Everyone reveals their cards at once.
- The highest and lowest estimators explain their reasoning. This is to learn what they were thinking, not to challenge them.
- After a short discussion everyone estimates again. Repeat until the numbers are close; it seldom takes more than three rounds.
- Settle on a reasonable figure. Exact agreement is not required.

The customer attends to clarify stories but does not estimate and does not react to the numbers.

Keeping estimates consistent

- **Triangulate:** compare each new estimate with stories already sized. A four-point story should look about twice a two-point one. Pinning cards to a wall in columns by size makes this easy.
- **Use a fixed scale.** Because precision falls as size rises, limit estimates to a set of values that widen as they grow (the book suggests ½, 1, 2, 3, 5, 8, 13, 20, 40, 80). Nobody then argues about a 7 versus an 8.

Velocity and why rough estimates still work

Velocity is the number of points a team completes in an iteration, and the best forecast for the next iteration is what was achieved in the last. Individual estimates may be badly off, but across an iteration's worth of independent stories the errors tend to even out. A measured velocity is a fair

predictor provided nothing unusual affected the iteration (overtime, extra staff), the estimates were produced consistently, and the stories were independent.

Reminders

- Points are not comparable between teams.
- When a story is split, the parts need not add up to the original estimate. The same goes for tasks.
- Pair programming does not change how you estimate. Whatever unit you choose, the effect of pairing shows up in velocity.

Takeaway Estimate size, not dates. Relative points plus measured velocity give a forecast that corrects itself.

Chapter 9. Planning a Release

In brief: a release plan allocates prioritised, estimated stories to iterations. It is a forecast to be revised, not a commitment to a date.

Most projects do well releasing every two to six months. Planning often begins from a roadmap listing the main themes of the next few releases, and then turns on two questions: when is the release wanted, and what is the priority of each story?

When

A range of acceptable dates is better than a single date. With stories it is easy to hit a date but hard to guarantee exactly what will be ready by then. A range lets the team say, for instance, that the minimum will take about so many iterations and the full wish list somewhat more.

What: prioritising

- Sorting into high, medium and low invites arguments about the boundaries. The **MoSCoW** rules from DSDM are a useful alternative: *must have*, *should have* (important, but a short-term workaround exists), *could have* (dropped if time runs out), *won't have this time*.
- The chapter's summary goes further: the end result should be a specific order (first, second, third), not just groups.
- Developers contribute technical factors: the risk that a story cannot be done as hoped, and the effect on other stories of putting it off.
- The customer weighs how many users want it, how much a few important users want it, and whether it belongs with other stories.
- The customer needs the estimates to prioritise, because cost changes desirability.
- If a story is hard to rank, it probably mixes parts of different importance. Split it.

"When there is a disagreement to the sequence, the customer wins. Every time."

— p. 99

Risk and infrastructure

- Older methods disagreed on whether to tackle the riskiest parts first (Boehm's spiral model) or the most valuable parts first (Gilb's "juicy bits"). Agile methods side with value first, which avoids building infrastructure that may never be needed and allows early release.

- Risk still counts. Developers must tell the customer which stories are risky or will become much more expensive if delayed. The customer then decides.
- Developers must not use this as an excuse to push their favourite technical work to the front.
- When an architectural need has no story behind it, write one that makes its value visible to the customer, so that it can be prioritised like anything else.

Iteration length

Choose one to four weeks and keep it constant so the project settles into a rhythm. Shorter iterations give more chances to correct course and more visibility, at a small overhead. If in doubt, go shorter. An occasional planned exception, for example to line up with a trade show, is fine; random changes are not.

From points to duration

Divide the total points by velocity to get the number of iterations. There are three ways to get a first velocity figure:

- Use the team's history, if the same team has just finished similar work.
- Run one iteration and measure.
- Make a reasoned guess. If a point is an ideal day, a common starting assumption is that velocity will be between a third and a half of the developer-days in an iteration.

Worked example in my own numbers: four developers and two-week iterations give 40 developer-days. A starting velocity of about 13 to 20 points is plausible. A 120-point release would then take roughly six to nine iterations.

Building and sharing the plan

Fill the first iteration with the highest-priority stories up to the velocity, then the second, and so on. Share it in whatever form suits the audience: cards on a wall in columns, a sorted spreadsheet with lines between iterations, photocopied cards, or a simple chart for stakeholders who expect one.

Takeaway A release plan supports statements like “about five to seven iterations”. It cannot support a specific calendar date. Reset expectations as velocity becomes known.

Chapter 10. Planning an Iteration

In brief: at the start of each iteration the whole team, with the customer, turns the next stories into tasks and checks that the workload is realistic.

The meeting, step by step

- **Discuss a story.** The customer reads the top-priority story. Developers ask enough to break it down, not every detail; finer points are settled during the iteration. This is also the moment for the customer to announce changed priorities.
- **Break it into tasks.** Someone writes them on a whiteboard as people call them out.
- **Accept responsibility.** One developer puts their name against each task.
- Repeat for further stories until the points discussed match the team's velocity.
- **Estimate and confirm.** Each developer estimates their own tasks in ideal time and judges whether the total is achievable.

Why break stories into tasks at all

- Several people often work on one story, by specialism or simply for speed.
- A story describes value to a user; it is not a to-do list. Listing tasks as a group catches things one person would forget, such as updating the installer or the help text.
- It is a short burst of just-in-time design, one of many that replace the single up-front design phase of a waterfall project.

Guidelines for tasks

- Separate out any task that is hard to estimate, for instance one that depends on a slow outside decision.
- Split work that different people could do in parallel.
- Make a separate task of anything it is useful to know is finished, so others can build on it.
- There is no required task size.

Responsibility is shared

A single name goes on each task even when pairing, and that person sees it through. But the commitment is the team's: if someone falls behind, others pick up the work. Nobody gets to say that their part is done while the iteration fails. If a developer finds they have taken on too much, another developer takes some of it, or the customer removes or splits a story. Simply hoping is the third option, and not a good one.

A note on changing priorities mid-iteration

The customer should hold new ideas until the next planning meeting. Halfway through building a feature, a better idea is being compared with nothing; at the end of the iteration it is compared with a working feature, and the comparison often comes out differently.

Takeaway Release planning says roughly which stories land in which iteration. Iteration planning says who does what in the next few weeks, and tests whether it fits.

Chapter 11. Measuring and Monitoring Velocity

In brief: measure velocity strictly, chart it visibly, and use the charts to decide whether the plan needs to change.

Rules for counting

- Count only stories that are completely finished, meaning they pass their acceptance tests.
- Partly finished stories count for nothing. Estimating a percentage complete is unreliable, incomplete work has no value to users, and the last tenth of a story can hide a lot of difficulty.
- If partial stories regularly make a difference to velocity, the stories are too big. If many stories end an iteration half-done, the team is not working together; better to finish some and leave others unstarted.
- Use the points assigned before the iteration. Do not rescore a story afterwards because it turned out harder; use that knowledge to adjust estimates of future stories.
- Do not draw conclusions from one or two iterations. Early velocity is volatile.

Four charts

Chart	Shows	Use it to
Planned vs actual velocity, per iteration	Whether each iteration's output matched the plan.	Spot a deviation, without overreacting to a single good or bad iteration.
Cumulative story points	Total points completed to date against the plan.	Judge whether the release as a whole is ahead or behind.
Iteration burndown	Points remaining at the end of each iteration, including work added or re-estimated.	See net progress toward the release. The line can rise if scope grows faster than work is completed.
Daily burndown	Estimated hours of task work remaining on each day of the current iteration.	Manage the iteration in progress.

Reading a burndown chart

A burndown chart does not show how fast the team is working. A team might finish a great deal while the customer adds even more. Net progress in an iteration is the points completed, less any increase in estimates of remaining stories, less the points of new stories. For team speed, look at the velocity or cumulative chart.

Practical habits

- Track hours **remaining**, not hours spent. Remaining effort is what matters, and logging time spent feels like micromanagement.
- Have developers update their own remaining estimates on a shared whiteboard at the end of each day or when a task ends. Raising an estimate should be as acceptable as lowering it.
- Add a task mid-iteration only if it was overlooked and is needed for a story already in the iteration. New wishes wait for the next planning session.
- Make the charts large and put them where everyone walks past.
- Watching defects per story point helps show whether a rise in velocity is being bought with lower quality.

Takeaway Done means done. Chart the result where everyone can see it and let the trend, not one data point, drive changes to the plan.

Part III: Frequently Discussed Topics

This part compares stories with other requirements techniques, argues the case for stories, lists warning signs, shows how stories fit Scrum, and answers several recurring practical questions.

Chapter 12. What Stories Are Not

In brief: stories differ from formal requirements specifications, from use cases, and from interaction-design scenarios, in size, purpose and lifespan.

	IEEE 830-style specification	Use case	Interaction-design scenario
What it is	A hierarchical list of statements, each beginning “The system shall”.	A structured description of interactions between an actor and the system: a main success path plus extensions.	A detailed narrative of particular people using the product in a setting, with goals, actions and events.
Size next to a story	Much finer grained; many statements per story.	Usually larger. A story is roughly one path through a use case.	Larger still. One scenario may span several use cases.
Main contrast	Lists attributes of a solution, where a story expresses a user's goal. Cost is not visible until everything is written.	Written to record an agreement and kept for the life of the product. Prone to UI detail.	Far more detail, and detail of a different kind.

Against the big specification

- Such documents are slow to write, error-prone and tedious to read, so people skim them and miss the overall picture.
- They cannot be completed up front, because users change their minds once they see working software. Calling that a “change of scope” wrongly implies the scope was ever fully known, and that software is finished when it meets a list, not when it meets users' goals.
- A list of attributes can describe the wrong product entirely. The book's illustration: a list specifying an engine, four wheels and a steering wheel suggests a car, while the user's actual goals (cut the grass quickly and comfortably) call for a ride-on mower.
- The customer cannot see the cost of each item while writing the wish list. With stories, each one carries an estimate from the start.
- A document passed back and forth between departments, each rewriting it under a new name, is a sign that people are preparing to assign blame.

Stories and use cases

- **Scope:** stories are deliberately kept small enough to schedule.
- **Completeness:** a story together with its acceptance tests carries about the same information as a use case; the tests correspond to the use case's extensions.
- **Longevity:** use cases are permanent artefacts. Stories are not meant to outlive the iteration that implements them.
- **Purpose:** use cases document agreement between customer and developers. Stories support planning and act as prompts for conversation.
- **Origin:** use cases are a product of analysis. Stories are an aid to it.
- *Essential use cases* (Constantine and Lockwood), stripped of technology assumptions, come close to stories: each user intention could be read as one.

Takeaway Think about what users are trying to achieve, not about a list of features the product should have. No amount of up-front thinking fully specifies a non-trivial system.

Chapter 13. Why User Stories?

In brief: eight advantages of stories over other techniques, and three honest drawbacks.

Advantages

- **They emphasise talking.** Written words look precise but are not; a sentence can be parsed two ways, and words such as “should” are open to interpretation. Writing does help memory, but conversation gives a fast feedback loop and no false sense of exactness. Even perfectly worded requirements would not help much: users' views change as they learn, and perfect parts do not guarantee a coherent whole.
- **Everyone can understand them.** They are short, free of technical jargon, and always framed by value. Research cited in the book found that people recall events better when they are organised as stories.
- **They are the right size for planning.** Formal statements are too many and too tangled to prioritise; use cases and scenarios are too big.
- **They suit iterative development.** You can start with an epic and refine it later, and you need not write them all before coding.
- **They encourage deferring detail,** so a team under time pressure can start sooner.
- **They support opportunistic development.** Studies show that designers do not work strictly top-down; they move between requirements, usage and design as opportunities appear. Stories allow that movement between levels.
- **They encourage participatory design.** Because stories are accessible, users become active contributors to deciding how the software behaves. This contrasts with *empirical design*, in which designers study users but decide alone.
- **They build tacit knowledge.** Frequent face-to-face discussion spreads understanding across the team.

Drawbacks

- On a large project with very many stories, it is hard to see how they relate. Using roles and keeping stories at a higher level until needed helps. Use cases have a built-in hierarchy that stories lack.
- If the process demands traceability from requirements to tests, extra documentation is needed, though it can be light.
- Conversation does not scale to very large teams; some things must be written. There is a trade-off between many people knowing a little from documents and fewer people knowing a lot from discussion.

Takeaway The aim is shared understanding, not a shared document. Know why you chose stories, and be ready to use another technique where it serves better.

Chapter 14. A Catalog of Story Smells

In brief: nine warning signs that stories are being misapplied, each with a remedy.

Smell	How you notice it	Remedy

Stories too small	Estimates keep needing revision, because the size of one story depends on which related story was done first.	Combine them for planning; split again only when scheduling into an iteration.
Interdependent stories	Iterations are hard to plan because stories drag other stories in with them.	Combine if they are too small. Otherwise re-split so each is a complete vertical slice.
Goldplating	Developers add unrequested features or interpret stories generously.	Increase visibility: brief daily meetings, end-of-iteration demonstrations, and QA involvement in story conversations.
Too many details	More time goes on writing about stories than discussing them.	Rely on conversation. Tom Poppendieck's advice, quoted below, is to shrink the card.
UI detail too soon	Early stories mention particular screens or controls.	Rewrite to describe what the user does, without assuming a layout.
Thinking too far ahead	Cards overflow; someone proposes a detailed template or finer-grained estimates.	Recall why the team chose stories: requirements cannot all be known in advance.
Splitting too many stories	Stories are routinely split during iteration planning to make things fit.	Occasional splitting is normal. If frequent, review the remaining stories and split in advance.
Customer has trouble prioritising	Ranking is unusually hard.	Make stories smaller so parts can be chosen separately, and rewrite any that do not express business value.
Customer will not write or prioritise	The customer avoids responsibility, often in a culture of blame.	Find a low-risk way for them to give opinions, in private if need be, and take responsibility for the final decision yourself.

"If you run out of room, use a smaller card."

— Tom Poppendieck, quoted on p. 159

Takeaway Both customer and developers are responsible for noticing these signs and acting on them.

Chapter 15. Using Stories with Scrum

In brief: stories came from Extreme Programming but work well as the requirements approach inside Scrum.

Iterative and incremental

The chapter separates two words that are often blurred. An **iterative** process improves a rough first version by repeated refinement, as a sculptor shapes the whole figure and then adds detail. An **incremental** process builds and delivers the product in finished pieces, as if the sculptor completed one part before starting the next. Scrum and XP are both.

Scrum as the book describes it

- Work proceeds in **sprints**, which the book gives as thirty days long.
- The **product backlog** is the prioritised master list of everything wanted. It is never complete and changes as people learn.
- The **sprint backlog** is the task-level list of what the team has committed to for the current sprint.
- The team is typically four to seven developers, organises itself, and owns all work collectively.
- The **product owner** corresponds to XP's customer and prioritises the product backlog.
- The **ScrumMaster** resembles a project manager but leads and serves instead of directing: removing obstacles and helping the team keep to Scrum's rules.

The three meetings

- **Sprint planning.** In the first half the product owner describes the top-priority items; in the second the team decides how much it can commit to and expands those items into tasks. Together they set a sprint goal. The team, not the product owner, decides how much to take on.
- **Daily scrum.** A short meeting, usually fifteen minutes or less, in which each person says what they did yesterday, what they will do today, and what is in their way. Problems are noted and solved afterwards. Observers may attend but only people on the project speak. It is not a status report to the ScrumMaster, and estimates are not collected there.
- **Sprint review.** An informal demonstration of working software, with no slides and little preparation, judged against the sprint goal.

Once a sprint has started, nobody outside the team may add work to it. In return for the team's commitment, the organisation agrees to leave the sprint alone. If something truly forces a change, the sprint is abandoned and a new one planned. Every sprint must end with a **potentially shippable product increment**: coded, tested and usable, whether or not the organisation chooses to release it.

What stories add

- Restricting the product backlog to stories means every item describes value the product owner can understand, which makes trading one item for another far easier.
- Role modeling followed by story writing gives Scrum a structured way to fill the backlog, which Scrum itself does not prescribe.
- Sprint planning goes faster because nobody has to explain the benefit of a purely technical item, and stories break down into tasks readily.
- Sprint reviews are simpler because items of user value are easier to demonstrate than a mixed bag of technical tasks.
- During the sprint, stories keep attention on what the user wanted, helping the team judge whether it has done enough or too much.

The case study

The chapter ends with a product that was rebuilt after a company acquisition. The original had been developed by a waterfall team of up to 100 people in nine months, about 540 person-

months. A Scrum team that never exceeded seven people rebuilt the same functionality using stories in twelve months, about 54 person-months, and with slightly less code.

Note for today's reader (mine, not the book's): Scrum has been revised several times since 2004. The current Scrum Guide allows sprints of one month or less, and many teams use two weeks. Check the current guide for present-day terminology and rules.

Takeaway Express every backlog item as a story. It keeps the backlog in the product owner's language and makes planning and review easier.

Chapter 16. Additional Topics

In brief: five practical questions that come up on most story-based projects.

1. Nonfunctional requirements

Not everything fits a story. Many nonfunctional needs (performance, accuracy, portability, reusability, maintainability, interoperability, availability, usability, security, capacity) are best treated as **constraints**: written on a card marked as such and, where practical, checked by an automated test that runs at least daily. Anything more complex should be documented in whatever form suits it, such as a data dictionary.

2. Paper cards or software?

In favour of cards	In favour of software
Simple, and their low-tech nature is a reminder that stories are imprecise.	Needed when the team or the customer is not in one place.
Limited space keeps stories short.	Helps when formal traceability is required, for example for ISO certification.
Easy to sort, stack, pin up and carry to meetings; they encourage discussion.	Stories do not get lost, and remote stakeholders can see progress.
	More room for acceptance tests than the back of a card.

Cohn's advice: start with cards, and switch only when there is a strong reason.

3. Stories and the user interface

Agile methods assume a system can be refined iteration by iteration. That holds beneath the surface, but changing an interface people have already learned is costly for them. If the interface matters to the product's success, think about it from the start. The book adapts Constantine and Lockwood's agile usage-centred design into eight story-based steps:

- Model the user roles.
- Trawl for high-level stories (about two dozen at most).
- Prioritise them.
- Refine the high- and medium-priority ones into plannable stories.
- Group stories likely to be performed together.
- Sketch a paper prototype for each group.

- Show it to users or proxies and refine it.
- Start programming.

Keep this light, a few days to a few weeks at most, since some of those stories may be dropped before they are built.

4. Keep or discard finished stories?

Tearing up a finished card is satisfying, but Cohn recommends keeping stories. Archived stories have proved useful to him when showing an acquiring company how his team worked, when rewriting a product years later, and when a prospective partner asked to see "the requirements".

5. Bugs

Treat a bug report as a story. A large fix is scheduled like any other story; small ones are bundled under a cover card and planned as one. Colour-coding cards by type is, in his view, more trouble than it is worth.

Takeaway Match the tool to the project. Use constraints for system-wide qualities, and give the interface early attention when it is central to the product.

Part IV: An Example

Chapters 17 to 21 follow one small invented project from first meeting to acceptance tests. A long-established mail-order seller of sailing supplies wants to start selling online, beginning with books only, and wants the site live in thirty days. A vice president of sales and marketing, Lori, acts as customer. The example is useful mainly for seeing the techniques used in sequence, so the notes below record what each chapter demonstrates.

Chapter 17. The User Roles

In brief: the role-modeling steps from Chapter 3, applied.

- Because the real buyers are outside the company, an internal person must act as proxy customer.
- The team brainstorms a dozen role cards, lays them out by overlap, and consolidates. Near-duplicates are merged, a role naming an organisation is replaced by one naming a person, and a role tied to a job title is renamed more generically.
- Eight roles remain: four kinds of sailor distinguished by experience and buying habits, a gift buyer who does not sail, a librarian, an administrator, and a viewer of management reports.
- Each card gets notes on frequency of use, domain knowledge, computer skill and goals.
- After asking the customer which roles matter most to success, the team writes two personas. The author adds that for a project this small personas are probably not worth the effort, and are included to complete the illustration.

Chapter 18. The Stories

In brief: a story-writing workshop, working through one role or persona at a time.

- The session yields 27 cards, covering search, book details, a shopping cart, checkout, ratings and reviews, accounts, wish lists, order history, recommendations, gift options, reports and administration.
- Open questions are left for later. When developers wonder exactly what a story means, they note it and keep writing.
- When discussion drifts into designing a feature, the team stops and writes a story instead.
- Known specifics go on the card in brackets, and uncertain ones are marked with a question mark.
- Three cards are constraints: a time limit for a returning customer to find and buy a book, a requirement to share the existing order database, and a concurrent-user capacity. The time limit is framed around the whole task the user performs, not one technical operation, which makes it a better performance requirement.
- A vague placeholder is acceptable for something not yet thought through, such as reporting.
- Finding every story is not the goal; finding enough for a preliminary estimate is. Later additions can be swapped in for work of similar size.

Chapter 19. Estimating the Stories

In brief: the team estimation method from Chapter 8, applied by three programmers with the customer present.

- Estimating forces clarifying questions. Asking what the customer meant by one search story reveals two modes, so the card is torn up and replaced by two.
- A first round gives three different numbers; after the high and low estimators explain, the second round converges.
- The customer listens and clarifies, and may point out that she is asking for something simpler than the developers assume, but does not estimate or show surprise.
- Stories are estimated as independent: a programmer's assumption that one will be quicker because another is done first is challenged, since the order is not yet known.
- Several stories are split because parts could be prioritised separately, which gives the customer finer control.
- A vague story is rewritten once the customer explains what she has in mind.
- Constraints get an estimate of zero. They shape other stories but need no code of their own.
- Estimates range from half a point to eight, the largest being the loosely defined reporting story.

Chapter 20. The Release Plan

In brief: the four planning steps from Chapter 9: choose an iteration length, estimate velocity, prioritise, allocate.

- With four weeks available, the team chooses two-week iterations, which gives two iterations and one chance to measure velocity and adjust.
- There is no comparable past project, so velocity is an educated guess. Two programmers over ten working days is twenty programmer-days. Assuming two to three real days per ideal day gives a range of seven to ten points, and they cautiously pick eight.

- The customer sorts cards into the four MoSCoW piles, considering cost as well as value.
- The must-have stories total nine points. Two iterations offer sixteen, so she adds should-have stories worth six and a half, for a total of fifteen and a half.
- The plan is published as two lists. If the first iteration goes better or worse than expected, stories move between iterations by agreement with the customer.

Chapter 21. The Acceptance Tests

In brief: the customer writes tests for each planned story, helped by the programmers because the project has no dedicated tester.

- Writing tests prompts more conversation and exposes unanswered questions, for example what happens with items that are out of stock.
- It can simplify the product. The team realises that a "remove from cart" story is a special case of changing a quantity, and replaces it with one more general story.
- Tests record decisions, such as which payment cards are accepted, by including both passing and failing cases.
- Tests cover permissions (what a non-administrator must not be able to do) and side effects (what happens to open orders when a book is deleted or repriced).
- Each constraint gets a concrete test: checking that an order placed online appears in the shared database, and simulating the stated number of simultaneous users with a maximum screen response time.

Takeaway Read Part IV to see the sequence: roles, then stories, then estimates, then a plan, then tests. Stories keep being split, merged and reworded at every step.

Part V: Appendices

Appendix B of the book gives answers to the end-of-chapter questions and is not summarised here.

Appendix A. An Overview of Extreme Programming

In brief: stories originated in Extreme Programming (XP). This appendix outlines its roles, practices, values and principles for readers new to it.

Roles

- **Customer:** writes and prioritises stories and writes and runs acceptance tests. May be one person or a team that speaks with one voice.
- **Programmer:** a broad role with no sharp division between coder, designer and database specialist. Programmers write automated unit tests for everything.
- **Coach** (and sometimes a project manager): keeps the team true to the practices and clears obstacles.

The twelve practices

Practice	Meaning

Small releases	Short, fixed-length iterations, each delivering complete, tested features. Scope flexes; the end date and quality do not.
The planning game	XP's name for release and iteration planning with estimated story cards.
Refactoring	Continually improving the structure of code without changing its behaviour. Required, not optional.
Testing	Programmers write automated unit tests, ideally before the code (test-driven development). Customers write acceptance tests.
Pair programming	Two programmers at one keyboard, swapping often. A study cited in the book found about 15% more programming time bought fewer defects and wider knowledge of the code.
Sustainable pace	Work at a brisk rate the team can keep up indefinitely.
Team code ownership	Anyone may change any code, with the unit tests as a safety net.
Coding standard	Shared conventions, needed because ownership is shared.
Simple design	The simplest design that delivers what is needed now: clear intent, no duplication, as few classes and methods as possible.
Metaphor	A shared image of how the whole system works.
Continuous integration	Integrate and run the tests at least daily, ideally after every small change.
On-site customer	The customer sits with the team to answer questions. Stories depend on this.

Values and principles

- **Four values:** communication, simplicity, feedback, courage.
- **Five principles:** rapid feedback, assume simplicity, incremental change, embrace change, quality work.
- Adopt all the practices before customising, since they reinforce one another. Cohn adds that a team living by the principles is doing XP even if it omits one or two.

Quick reference

Glossary

Term	Meaning in the book
User story	A short description of functionality valued by a user or purchaser, made up of a card, a conversation and a confirmation.
Epic	A story too large to estimate or build as it stands. Useful as a placeholder.
Compound / complex story	Two kinds of epic: one that contains several smaller stories, and one that is inherently large or uncertain.

Spike	A short, time-boxed investigation done to learn enough to estimate or build something.
Constraint	A rule the system must obey, such as a performance or capacity target. Recorded on a card but not scheduled like a story.
User role	A group of users defined by shared characteristics and by how they will use the system.
Persona	An invented, named individual who represents an important user role.
User proxy	Someone who stands in for real users on the project.
Customer team	The people who make sure the product meets users' needs: users, product manager, testers, interaction designers.
Acceptance test	A test, specified by the customer, that shows a story works as intended.
Story point	A team-defined, relative unit for the size of a story.
Ideal day	A working day with no interruptions. A common basis for story points.
Velocity	Story points completed in an iteration. Only finished stories count.
Iteration	A fixed period, usually one to four weeks, ending with working, tested software.
Release plan	A rough allocation of prioritised stories to iterations.
Triangulation	Checking an estimate by comparing it with stories already estimated.
MoSCoW	Prioritisation into must have, should have, could have, and won't have this time.
Burndown chart	A graph of work remaining over time, by iteration (points) or by day (hours).
Goldplating	Developers adding features nobody asked for.
Product backlog / sprint backlog	In Scrum, the prioritised list of everything wanted, and the task list for the current sprint.

Who is responsible for what

Most chapters end with separate lists of responsibilities for developers and for the customer. Condensed:

Activity	Customer	Developers
Writing stories	Writes them, in business language, at a suitable size.	Help, and reword technical needs in terms of user value.
User roles	Looks broadly across possible users and avoids focusing on a subset.	Take part, and keep each role in mind while building.
Finding stories	Uses several techniques and makes sure every role is represented.	Learn to ask open-ended, context-free questions.
Proxies	Knows what kind of proxy they are and what bias that brings.	Help the organisation choose a suitable customer.

Acceptance tests	Specifies and runs them.	Automate them where chosen, suggest more, and unit test the code.
Estimating	Answers questions. Does not estimate.	Estimate together, honestly and consistently.
Release planning	Puts stories in a precise order, gives real deadlines, and splits stories with mixed priorities.	Supply estimates, risks and alternatives without over-promoting technical work.
Iteration planning	Directs the team to the most valuable work and attends the meeting.	Break all stories into tasks, sign up, and take on a realistic load.
Tracking	Knows the team's velocity and adds or removes stories to meet the goals.	Finish one story before starting the next; keep remaining-work estimates current.

A checklist for one story

My own condensed checklist, drawn from Chapters 2, 6, 7 and 14.

- Does it name a specific role, in the singular and in the active voice?
- Would a user or purchaser care if it were missing?
- Can it be built without waiting for another story?
- Could the team estimate it today? If not, is the gap domain knowledge, technology, or size?
- Does it fit easily inside one iteration?
- Is it free of screen and technology detail that could be decided later?
- Does finishing it leave the user having accomplished something?
- Are there a few test reminders on the back that would show it is done?
- Is anything on the card a detail that should be a test, or a second story?

Source

Cohn, Mike. *User Stories Applied: For Agile Software Development.* Addison-Wesley Signature Series. Boston: Addison-Wesley, 2004. ISBN 0-321-20568-5. Page numbers given with quotations refer to the printed edition.